8. Symmetry Groups of Boolean Functions and Circuits

This section studies symmetry as a structural invariant of Boolean functions and the circuits that realize them. The central objects are actions of permutation groups on input variables, induced transformations on truth tables and circuit descriptions, and the resulting equivalence relations used to classify functions up to relabeling, negation, and output complementation. In this setting, symmetry is not merely an aesthetic property: it is an algorithmic handle for canonicalization, equivalence checking, and the extraction of compressed representations.

Let $f : \mathbb{B}^n \to \mathbb{B}$ be a Boolean function. The symmetric group Σ_n acts on $\mathbb{B}^n$ by permuting coordinates, and this action induces a corresponding action on functions by precomposition. A permutation $\pi \in \Sigma_n$ is an automorphism of f when

$$f(x_1, \ldots, x_n) = f(x_{\pi(1)}, \ldots, x_{\pi(n)})$$

for all inputs $x \in \mathbb{B}^n$. The set of all such permutations forms the automorphism group $\mathrm{Aut}(f) \leq \Sigma_n$. More generally, the stabilizer of a function under a chosen group action records the symmetries that preserve the function exactly, while orbits partition the space of functions into equivalence classes under that action. These orbit–stabilizer notions are the basic language for symmetry-aware classification.

For circuits, the same viewpoint applies at the level of wiring diagrams and gate labels. A circuit automorphism may permute input pins, internal wires, or structurally identical subcomponents while preserving the computed function or the circuit graph up to isomorphism. This distinction matters: two circuits can compute the same Boolean function while having different internal symmetries, and conversely a circuit may exhibit graph-level symmetry that is not visible in the truth table alone. The book therefore treats function symmetry and circuit symmetry as related but not identical layers of structure. In practice, circuit-level symmetry information can be used to prune search, identify repeated substructures, and support canonical labeling of netlists.

A particularly important equivalence relation is NPN equivalence. Two Boolean functions f and g are NPN-equivalent if one can be obtained from the other by a combination of input negations, input permutations, and output negation. Concretely, there exist a permutation $\pi \in \Sigma_n$, a vector $a \in \mathbb{B}^n$ specifying input complements, and a bit $b \in \mathbb{B}$ specifying output complement such that

$$g(x) = f(x_{\pi(1)} \oplus a_1, \ldots, x_{\pi(n)} \oplus a_n) \oplus b.$$

NPN equivalence is a standard classification notion in logic synthesis because it identifies functions that differ only by renaming and polarity changes of variables. Canonical forms for NPN classes provide a representative for each orbit, enabling database indexing, pattern matching, and symmetry-aware optimization. In practice, a canonicalizer must balance exactness, speed, and robustness on large truth tables or circuit instances. A useful canonicalizer therefore combines structural invariants, orbit pruning, and tie-breaking rules that are stable under repeated application.

The spectral perspective supplies a complementary set of tools. Writing the Walsh–Hadamard transform of f as $\widehat{f}(S)$ for subsets $S \subseteq \{1, \ldots, n\}$, one obtains a frequency-domain representation that is sensitive to variable interactions and often easier to compare under symmetry. Approximate symmetry can be quantified by a spectral correlation score $\varepsilon(r)$, where r denotes a candidate relabeling or transformation and the score measures how closely the transformed spectrum matches the original. In this book, $\varepsilon(r)$ serves as a practical heuristic for detecting near-symmetries, ranking candidate automorphisms, and guiding search before exact verification. Spectral methods are especially useful when exact group computation is expensive, or when the object under study is

noisy, incomplete, or only approximately symmetric. They also provide a natural bridge between exact symmetry and approximate invariance: a high correlation score suggests a plausible symmetry, while a low score can eliminate candidates early.

Beyond permutation symmetries, linear and affine transformations provide a broader algebraic framework. The general linear group $\mathrm{GL}(n, 2)$ acts on $\mathbb{B}^n$ by invertible linear maps over $\mathbb{F}_2$, and the affine group $\mathrm{AGL}(n, 2)$ extends this action by translations. These actions are relevant when one studies functions up to linear or affine equivalence, a coarser but often computationally tractable relation. Spectral methods interact naturally with these transformations because linear changes of variables reorganize Fourier coefficients in structured ways. This makes the Walsh–Hadamard domain a useful arena for both exact invariants and approximate matching under affine symmetries. In particular, spectral signatures can be used to cluster functions before exact orbit computation, reducing the cost of downstream canonicalization.

A practical symmetry-analysis workflow has two stages. First, a symmetry oracle proposes candidate automorphisms, stabilizers, or equivalence witnesses using structural cues, orbit information, and spectral signatures. Second, a canonicalizer normalizes the function or circuit to a chosen representative, producing a reproducible form suitable for comparison across instances. The oracle is intentionally permissive: it may return false positives, but it should be fast and high-recall. The canonicalizer is intentionally strict: it must verify the chosen representative and ensure that repeated application is idempotent. Benchmarks are used to evaluate both correctness and performance, with particular attention to the tradeoff between exhaustive group search and heuristic pruning. A practical benchmark suite should report not only runtime, but also orbit sizes discovered, canonical-form stability, and the rate at which spectral candidates are confirmed by exact checks.

For implementation, it is useful to separate three levels of output. At the first level, the system records invariants such as support size, algebraic degree, and coarse spectral statistics. At the second level, it enumerates candidate symmetries and computes stabilizer information for the most promising candidates. At the third level, it emits a canonical label or normal form together with a proof trace or witness certificate when available. This stratification makes the artifact easier to test and easier to integrate into synthesis or verification pipelines. It also clarifies where approximation is allowed: the oracle may be heuristic, but the canonicalizer and any reported automorphism should be exact.

The main conceptual thread is that symmetry is a closure phenomenon: once a function or circuit is placed under a group action, its meaningful invariants are those that survive the action, and its classification is governed by orbit structure. Automorphisms, stabilizers, and canonical forms provide the exact algebraic language; NPN equivalence provides the synthesis-oriented classification; and spectral methods provide scalable approximations and diagnostics. Together, these tools prepare the ground for later chapters on algorithmic canonicalization, equivalence checking, and symmetry-aware design. They also establish a reusable pattern for the rest of the book: identify the acting group, compute or approximate its orbits, and extract canonical representatives that support comparison, reuse, and verification.